

Data Analysis to Answer Questions

Session 5

Agenda

- **Combine Multiple Data Sources in Python**

Introduction to Data Merging in Python

- **Data merging** is a critical step in data analysis, allowing analysts to integrate data from various sources to form a comprehensive dataset.
- Using Python's pandas library, we can concatenate, merge, and join datasets efficiently, paving the way for insightful analysis.

Pandas Data Structures for Merging

Before merging, it's essential to understand the primary data structures in pandas:

- **DataFrame:** A 2-dimensional labeled data structure with columns of potentially different types.
- **Series:** A one-dimensional array with labeled indexing.

DataFrames are pivotal when it comes to merging different data sources as they easily allow for operations like merge, join, and concatenate.

Concatenating DataFrames

Concatenation is the simplest form of data combination, stacking data from multiple sources vertically or horizontally.

- **Vertical Concatenation:** Combining rows from different DataFrames.
- **Horizontal Concatenation:** Combining columns from different DataFrames.

```
import pandas as pd
df1 = pd.DataFrame({'A': ['A0', 'A1'], 'B': ['B0', 'B1']})
df2 = pd.DataFrame({'A': ['A2', 'A3'], 'B': ['B2', 'B3']})
result = pd.concat([df1, df2], axis=0) # Change axis to 1 for horizontal
print(result)
```

Output

	A	B
0	A0	B0
1	A1	B1
0	A2	B2
1	A3	B3

Categories of Joins

The **pd.merge** function implements a number of types of joins: **one-to-one**, **many-to-one**, and **many-to-many**. All three types of joins are accessed via an identical call to the **pd.merge** interface; the type of join performed depends on the form of the input data. We'll start with some simple examples of the three types of merges and discuss detailed options a bit later.

One-to-One Joins

- Perhaps the simplest type of merge is the one-to-one join, which is in many ways similar to the column-wise concatenation. As a concrete example, consider the following two DataFrame objects, which contain information on several employees in a company

df1

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

df2

	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014

One-to-One Joins

- To combine this information into a single DataFrame, we can use the `pd.merge` function

```
df3 = pd.merge(df1, df2)  
df3
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

One-to-One Joins

- The `pd.merge` function automatically joins two DataFrames using a common column (like "employee") as a key.
- It combines information from both DataFrames, even if the column order differs. The merge does not maintain the original index unless specified using `left_index` and `right_index` options.

```
df3 = pd.merge(df1, df2)  
df3
```

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

Many-to-One Joins

Many-to-one joins are joins in which one of the two key columns contains duplicate entries. For the many-to-one case, the resulting DataFrame will preserve those duplicate entries as appropriate. Consider the following example of a many-to-one join:

```
df4 = pd.DataFrame({'group': ['Accounting', 'Engineering', 'HR'],
                    'supervisor': ['Carly', 'Guido', 'Steve']})
display('df3', 'df4', 'pd.merge(df3, df4)')
```

df3

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

df4

	group	supervisor
0	Accounting	Carly
1	Engineering	Guido
2	HR	Steve

pd.merge(df3, df4)

	employee	group	hire_date	supervisor
0	Bob	Accounting	2008	Carly
1	Jake	Engineering	2012	Guido
2	Lisa	Engineering	2004	Guido
3	Sue	HR	2014	Steve

Many-to-Many Joins

A many-to-many join occurs when both DataFrames contain duplicate values in the key column. This results in multiple matches for each key. For example, if one DataFrame lists people and another lists their skills, a many-to-many join will combine the data to show all the skills associated with each person.

df1

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

df5

	group	skills
0	Accounting	math
1	Accounting	spreadsheets
2	Engineering	software
3	Engineering	math
4	HR	spreadsheets
5	HR	organization

pd.merge(df1, df5)

	employee	group	skills
0	Bob	Accounting	math
1	Bob	Accounting	spreadsheets
2	Jake	Engineering	software
3	Jake	Engineering	math
4	Lisa	Engineering	software
5	Lisa	Engineering	math
6	Sue	HR	spreadsheets
7	Sue	HR	organization

The on Keyword

- Most simply, you can explicitly specify the name of the key column using the on keyword, which takes a column name or a list of column names
- This option works only if both the left and right DataFrames have the specified column name.

```
pd.merge(df1, df2, on='employee')
```

df1

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

df2

	employee	hire_date
0	Lisa	2004
1	Bob	2008
2	Jake	2012
3	Sue	2014

pd.merge(df1, df2, on='employee')

	employee	group	hire_date
0	Bob	Accounting	2008
1	Jake	Engineering	2012
2	Lisa	Engineering	2004
3	Sue	HR	2014

The left_on and right_on Keywords

- At times you may wish to merge two datasets with different column names; for example, we may have a dataset in which the employee's name is labeled as "name" rather than "employee". In this case, we can use the left_on and right_on keywords to specify the two column names:

```
pd.merge(df1, df3, left_on="employee", right_on="name")
```

df1

	employee	group
0	Bob	Accounting
1	Jake	Engineering
2	Lisa	Engineering
3	Sue	HR

df3

	name	salary
0	Bob	70000
1	Jake	80000
2	Lisa	120000
3	Sue	90000

pd.merge(df1, df3, left_on="employee", right_on="name")

	employee	group	name	salary
0	Bob	Accounting	Bob	70000
1	Jake	Engineering	Jake	80000
2	Lisa	Engineering	Lisa	120000
3	Sue	HR	Sue	90000

The left_index and right_index Keywords

```
df1a = df1.set_index('employee')  
df2a = df2.set_index('employee')
```

df1a

group	
employee	
Bob	Accounting
Jake	Engineering
Lisa	Engineering
Sue	HR

df2a

hire_date	
employee	
Lisa	2004
Bob	2008
Jake	2012
Sue	2014

The left_index and right_index Keywords

You can use the index as the key for merging by specifying the left_index and/or right_index flags in pd.merge():

```
pd.merge(df1a, df2a, left_index=True, right_index=True)
```

df1a

group

employee

Bob Accounting

Jake Engineering

Lisa Engineering

Sue HR

df2a

hire_date

employee

Lisa 2004

Bob 2008

Jake 2012

Sue 2014

pd.merge(df1a, df2a, left_index=True, right_index=True)

group hire_date

employee

Bob Accounting 2008

Jake Engineering 2012

Lisa Engineering 2004

Sue HR 2014

join() Method

- For convenience, Pandas includes the `DataFrame.join()` method, which performs an index-based merge without extra keywords:

```
df1a.join(df2a)
```

employee	group hire_date	
Bob	Accounting	2008
Jake	Engineering	2012
Lisa	Engineering	2004
Sue	HR	2014

The how Keyword

the intersection of the two sets of inputs; this is what is known as an **inner join**. We can specify this explicitly using the how keyword, which defaults to "inner"

```
pd.merge(df6, df7, how='inner')
```

	name	food	drink
0	Mary	bread	wine

The how Keyword

Other options for the how keyword are '**outer**', '**left**', and '**right**'. An outer join returns a join over the union of the input columns, and fills in all missing values with NAs:

```
pd.merge(df6, df7, how='outer')
```

df6

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread

df7

	name	drink
0	Mary	wine
1	Joseph	beer

```
pd.merge(df6, df7, how='outer')
```

	name	food	drink
0	Peter	fish	NaN
1	Paul	beans	NaN
2	Mary	bread	wine
3	Joseph	NaN	beer

The how Keyword

- The left join and right join return joins over the left entries and right entries, respectively.

```
pd.merge(df6, df7, how='left')
```

df6

	name	food
0	Peter	fish
1	Paul	beans
2	Mary	bread

df7

	name	drink
0	Mary	wine
1	Joseph	beer

pd.merge(df6, df7, how='left')

	name	food	drink
0	Peter	fish	NaN
1	Paul	beans	NaN
2	Mary	bread	wine

- The output rows now correspond to the entries in the left input. Using how='right' works in a similar manner.
- All of these options can be applied straightforwardly to any of the preceding join types.

Overlapping Column Names: The suffixes Keyword

you may end up in a case where your two input DataFrames have conflicting column names. Consider this example:

```
pd.merge(df8, df9, on="name")
```

df8

	name	rank
0	Bob	1
1	Jake	2
2	Lisa	3
3	Sue	4

df9

	name	rank
0	Bob	3
1	Jake	1
2	Lisa	4
3	Sue	2

pd.merge(df8, df9, on="name")

	name	rank_x	rank_y
0	Bob	1	3
1	Jake	2	1
2	Lisa	3	4
3	Sue	4	2

Overlapping Column Names: The suffixes Keyword

Because the output would have two conflicting column names, the merge function automatically appends the suffixes `_x` and `_y` to make the output columns unique. If these defaults are inappropriate, it is possible to specify a custom suffix using the `suffixes` keyword:

```
pd.merge(df8, df9, on="name", suffixes=["_L", "_R"])
```

	name	rank_L	rank_R
0	Bob	1	3
1	Jake	2	1
2	Lisa	3	4
3	Sue	4	2

Merging Sales and Inventory Data Hand-on Exercise

Task: Consider a scenario where a retail chain needs to merge quarterly sales data with their inventory levels to analyze product performance.

Best Practices in Data Merging

- **Data Integrity:** Always verify the integrity of your data post-merge.
- **Duplicate Management:** Handle duplicates carefully, especially when merging large datasets.
- **Performance Considerations:** Be mindful of memory and time efficiency, especially with large-scale data operations.

Any Questions ?

Thank You

References

<https://colab.research.google.com/github/jakevdp/PythonDataScienceHandbook/blob/master/notebooks/03.07-Merge-and-Join.ipynb>

